CareOps

Smart Healthcare Operations & Treatment Analytics System

Project Type: DBMS + Data Warehousing

Semester: 4th Semester

Subjects: Database Management Systems & Data Warehousing

Estimated Project Rating: **8.8 / 10 → 9.5 / 10**

Contents

1 Introduction & Project Overview **3**
1.1 Project Title . . . 3
1.2 Project Rating . . . 3
1.3 What This Project Is (and Is NOT) . . . 4

2 Phase 0 — Scope Lock **5**
2.1 Stakeholders . . . 5
2.2 Business Questions the System Must Answer . . . 5

3 Phase 1 — Requirement Analysis **6**
3.1 Functional Requirements . . . 6
3.2 Non-Functional Requirements . . . 6

4 Phase 2 — OLTP Database Design **7**
4.1 Entity-Relationship Overview . . . 7
4.2 Full DDL — Create Tables . . . 7
4.3 Normalization Proof . . . 11
4.3.1 First Normal Form (1NF) . . . 11
4.3.2 Second Normal Form (2NF) . . . 11
4.3.3 Third Normal Form (3NF) . . . 11

5 Phase 3 — Advanced DBMS Features **12**
5.1 Trigger: Auto-Flag Long Hospital Stays . . . 12
5.2 Stored Procedure: Admit Patient . . . 12
5.3 View: Doctor Performance Summary . . . 13

6 Phase 4 — Data Simulation **15**
6.1 Data Volume Targets . . . 15
6.2 Python Data Generation Script . . . 15

7 Phase 5 — Data Warehouse Design **18**
7.1 Granularity Declaration . . . 18
7.2 Star Schema DDL . . . 18
7.2.1 Why Surrogate Keys? . . . 20

8 Phase 6 — ETL Pipeline **22**
8.1 ETL Architecture . . . 22
8.2 Step 1: Extract . . . 22

8.3 Step 2: Transform 23
8.4 Step 3: Load 23

9 Phase 7 — Analytics Queries **25**
9.1 Query 1: Disease Trend Over Time 25
9.2 Query 2: Doctor Efficiency Ranking 25
9.3 Query 3: Ward Utilization Rate 26
9.4 Query 4: Cost Analysis by Disease 26
9.5 Query 5: Readmission Rate by Disease 26

10 Phase 8 — Viva Preparation **28**
10.1 Expected Viva Questions with Model Answers 28

11 Final Honest Assessment **30**
11.1 Recommended Build Order 30

Chapter 1

Introduction & Project Overview

[!] Brutal Honesty

Most undergrad DBMS projects are shallow CRUD apps with 3 tables and a login screen. **CareOps is different.** It is a data systems project — OLTP + ETL + Data Warehouse + Analytics. That said, healthcare analytics is not a new domain. What makes this "elite" is that *you must implement it correctly*, which 80% of students fail to do.

1.1 Project Title

CareOps

Smart Healthcare Operations & Treatment Analytics System

1.2 Project Rating

Aspect	Stars	Comment
Database Necessity	★★★★★	Genuinely requires relational DB
Warehousing Fit	★★★★★	Star schema is the natural choice
Examiner Appeal	★★★★★	Familiar domain, clear metrics
Uniqueness (UG level)	★★★★○	Not novel, but correctly rare
Complexity Control	★★★★★	Scope is manageable
Overall (base)		**8.8 / 10**
Overall (with Outcome)		**9.5 / 10**

1.3 What This Project Is (and Is NOT)

[TIP] Pro Tip

Read this carefully before you code a single line.

IN SCOPE	OUT OF SCOPE
OLTP relational database (3NF)	Appointment booking UI or mobile app
Data warehouse (star schema)	Real patient data or PHI
ETL pipeline (Extract-Transform-Load)	Machine learning / diagnosis AI
Analytics queries (SQL)	IoT sensors or real-time streaming
Triggers & stored procedures	Authentication / role-based access systems
Simulated data (500+ patients)	Cloud deployment

[!] Brutal Honesty

This is a **data systems project**, not a medical app. If your examiner asks "where is the UI?", your answer is: "The deliverable is the analytics layer, which feeds any front-end. Business intelligence tools like Tableau or Power BI would sit on top of this warehouse." Say it confidently.

Chapter 2

Phase 0 — Scope Lock

Phase 0: Scope Lock — The Most Important Phase

This is written first for a reason. Scope creep kills undergraduate projects. Lock this now, and do not deviate.

2.1 Stakeholders

- **Hospital Administrator** — Resource allocation, ward occupancy, cost
- **Medical Superintendent** — Doctor performance, treatment outcomes
- **Operations Manager** — Bed utilization, patient throughput

2.2 Business Questions the System Must Answer

These questions are the *sole justification* for the data warehouse. Write them down, commit to them.

1. Which diseases consume the most hospital resources?
2. Which doctors achieve the best recovery outcomes?
3. Which wards are chronically over-utilized?
4. Are treatment costs increasing over time?
5. What is the hospital's patient readmission rate?

[TIP] Pro Tip

In your viva, when asked "Why did you build a data warehouse?", recite these five questions. A data warehouse is justified when: (a) queries span multiple entities, (b) aggregation is needed over time, and (c) OLTP performance would degrade under analytical load. All three apply here.

Chapter 3

Phase 1 — Requirement Analysis

3.1 Functional Requirements

1. The system shall store patient demographic data (anonymized).
2. The system shall record each patient visit, including assigned doctor and date.
3. The system shall store diagnoses with ICD-like disease codes and severity.
4. The system shall track treatments (medicines, dosage, duration).
5. The system shall manage ward and bed allocation.
6. The system shall record treatment outcomes (Recovered / Improved / Readmitted).
7. The data warehouse shall support aggregated queries over time dimensions.
8. ETL shall run periodically to refresh warehouse data from OLTP.

3.2 Non-Functional Requirements

- **Normalization:** OLTP tables in Third Normal Form (3NF)
- **Scalability:** System tested with 500+ patients, 2000+ visits
- **Integrity:** Referential integrity enforced via foreign keys
- **Auditability:** Triggers log anomalies automatically

Chapter 4

Phase 2 — OLTP Database Design

Phase 2: OLTP — The Core DBMS Component

This is graded most heavily. Get normalization right. Do NOT add columns for things you cannot populate with real (simulated) data.

4.1 Entity-Relationship Overview

The OLTP database has **9 entities**:

Entity	Primary Key	Purpose
Patient	patient_id	Stores anonymized patient demographics
Doctor	doctor_id	Stores doctor specialization and experience
Disease	disease_code	ICD-style disease catalog
Medicine	medicine_id	Drug catalog
Visit	visit_id	Central transaction — one visit per episode
Diagnosis	(visit_id, disease_code)	Links visit to disease(s)
Treatment	treatment_id	Medicine prescribed per visit
Ward	ward_id	Physical ward definitions
BedAllocation	allocation_id	Patient-to-ward assignments
Outcome	outcome_id	Post-visit treatment result

4.2 Full DDL — Create Tables

```
-- ============================================================
-- CareOps OLTP Database
-- DBMS: MySQL 8.x compatible
-- ============================================================

CREATE DATABASE IF NOT EXISTS careops_oltp;
```

```sql
USE careops_oltp;

-- ------------------------------------------------------------
-- PATIENT (anonymized, no PII)
-- ------------------------------------------------------------
CREATE TABLE Patient (
    patient_id      INT           AUTO_INCREMENT PRIMARY KEY,
    age_group       VARCHAR(20)   NOT NULL,    -- e.g. '18-30', '31-45'
    gender          CHAR(1)       NOT NULL     CHECK (gender IN ('M','F','O')),
    blood_group     VARCHAR(5),
    city            VARCHAR(50)
);

-- ------------------------------------------------------------
-- DOCTOR
-- ------------------------------------------------------------
CREATE TABLE Doctor (
    doctor_id         INT           AUTO_INCREMENT PRIMARY KEY,
    specialization    VARCHAR(80)   NOT NULL,
    experience_years  INT           NOT NULL CHECK (experience_years >= 0),
    department        VARCHAR(60)
);

-- ------------------------------------------------------------
-- DISEASE (ICD-style catalog)
-- ------------------------------------------------------------
CREATE TABLE Disease (
    disease_code  VARCHAR(10)  PRIMARY KEY,
    disease_name  VARCHAR(100) NOT NULL,
    category      VARCHAR(60)  NOT NULL,   -- e.g. Respiratory, Cardiac
    severity_tier VARCHAR(20)  NOT NULL   CHECK (severity_tier IN ('Low','Moderate','High','Critical'))
);

-- ------------------------------------------------------------
-- MEDICINE
-- ------------------------------------------------------------
CREATE TABLE Medicine (
    medicine_id    INT          AUTO_INCREMENT PRIMARY KEY,
    medicine_name  VARCHAR(100) NOT NULL,
    medicine_type  VARCHAR(50),            -- Antibiotic, Analgesic, etc.
    unit_cost      DECIMAL(10,2) NOT NULL CHECK (unit_cost >= 0)
);

-- ------------------------------------------------------------
-- WARD
```

```sql
-- ------------------------------------------------------------
CREATE TABLE Ward (
    ward_id     INT           AUTO_INCREMENT PRIMARY KEY,
    ward_name   VARCHAR(60)   NOT NULL,
    ward_type   VARCHAR(40)   NOT NULL,    -- ICU, General, Pediatric, etc.
    capacity    INT           NOT NULL CHECK (capacity > 0)
);

-- ------------------------------------------------------------
-- VISIT  (central transaction table)
-- ------------------------------------------------------------
CREATE TABLE Visit (
    visit_id      INT          AUTO_INCREMENT PRIMARY KEY,
    patient_id    INT          NOT NULL,
    doctor_id     INT          NOT NULL,
    visit_date    DATE         NOT NULL,
    visit_type    VARCHAR(20) NOT NULL CHECK (visit_type IN ('OPD','IPD','Emergency')),
    FOREIGN KEY (patient_id) REFERENCES Patient(patient_id),
    FOREIGN KEY (doctor_id)  REFERENCES Doctor(doctor_id)
);

-- ------------------------------------------------------------
-- DIAGNOSIS  (a visit can have multiple diagnoses)
-- ------------------------------------------------------------
CREATE TABLE Diagnosis (
    visit_id       INT          NOT NULL,
    disease_code   VARCHAR(10) NOT NULL,
    severity       VARCHAR(20) NOT NULL CHECK (severity IN ('Mild','Moderate','Severe')),
    PRIMARY KEY (visit_id, disease_code),
    FOREIGN KEY (visit_id)     REFERENCES Visit(visit_id),
    FOREIGN KEY (disease_code) REFERENCES Disease(disease_code)
);

-- ------------------------------------------------------------
-- TREATMENT  (medicines prescribed per visit)
-- ------------------------------------------------------------
CREATE TABLE Treatment (
    treatment_id    INT           AUTO_INCREMENT PRIMARY KEY,
    visit_id        INT           NOT NULL,
    medicine_id     INT           NOT NULL,
    dosage_mg       DECIMAL(8,2)  NOT NULL,
    duration_days   INT           NOT NULL CHECK (duration_days > 0),
    FOREIGN KEY (visit_id)    REFERENCES Visit(visit_id),
    FOREIGN KEY (medicine_id) REFERENCES Medicine(medicine_id)
);

-- ------------------------------------------------------------
```

```sql
-- BED ALLOCATION
-- ------------------------------------------------------------
CREATE TABLE BedAllocation (
    allocation_id   INT  AUTO_INCREMENT PRIMARY KEY,
    patient_id      INT  NOT NULL,
    ward_id         INT  NOT NULL,
    admit_date      DATE NOT NULL,
    discharge_date  DATE,
    days_stayed     INT  GENERATED ALWAYS AS
                      (DATEDIFF(discharge_date, admit_date))
                         STORED,
    FOREIGN KEY (patient_id) REFERENCES Patient(patient_id),
    FOREIGN KEY (ward_id)    REFERENCES Ward(ward_id)
);

-- ------------------------------------------------------------
-- OUTCOME  (** This addition raises the project to 9.5/10 **)
-- ------------------------------------------------------------
CREATE TABLE Outcome (
    outcome_id  INT         AUTO_INCREMENT PRIMARY KEY,
    visit_id    INT         NOT NULL UNIQUE,
    status      VARCHAR(20) NOT NULL
                  CHECK (status IN ('Recovered','Improved','
                     Readmitted','Deceased')),
    outcome_date DATE       NOT NULL,
    FOREIGN KEY (visit_id) REFERENCES Visit(visit_id)
);

-- ------------------------------------------------------------
-- ALERT LOG  (populated by trigger)
-- ------------------------------------------------------------
CREATE TABLE AlertLog (
    alert_id      INT           AUTO_INCREMENT PRIMARY KEY,
    patient_id    INT           NOT NULL,
    ward_id       INT           NOT NULL,
    days_stayed   INT,
    alert_message VARCHAR(255),
    created_at    TIMESTAMP     DEFAULT CURRENT_TIMESTAMP
);
```

Listing 4.1: OLTP Schema — careops_oltp database

[!] Brutal Honesty

Do NOT add columns like `doctor_name` inside the `Visit` table — that is a 2NF violation. Do NOT store derived values like `total_cost` in the OLTP Visit table — aggregate in the warehouse. If you cannot populate a column with simulated data, remove it.

4.3 Normalization Proof

4.3.1 First Normal Form (1NF)

All tables have atomic columns. No repeating groups. Every row is uniquely identified by its primary key.

4.3.2 Second Normal Form (2NF)

The only composite key is `Diagnosis(visit_id, disease_code)`.

- `severity` depends on the full composite key (same visit can diagnose same disease differently) — ✓ 2NF satisfied.

4.3.3 Third Normal Form (3NF)

No transitive dependencies exist.

- `doctor_id` → `specialization` lives in `Doctor` table, not `Visit`.
- `disease_code` → `disease_name` lives in `Disease` table, not `Diagnosis`.

All non-key attributes depend only on the primary key. **3NF confirmed.**

Chapter 5

Phase 3 — Advanced DBMS Features

5.1 Trigger: Auto-Flag Long Hospital Stays

```sql
DELIMITER $$

CREATE TRIGGER trg_long_stay
AFTER UPDATE ON BedAllocation
FOR EACH ROW
BEGIN
    IF NEW.days_stayed IS NOT NULL AND NEW.days_stayed > 14 THEN
        INSERT INTO AlertLog (patient_id, ward_id, days_stayed,
           alert_message)
        VALUES (
            NEW.patient_id,
            NEW.ward_id,
            NEW.days_stayed,
            CONCAT('Patient ', NEW.patient_id,
                   ' has stayed ', NEW.days_stayed,
                   ' days in ward ', NEW.ward_id,
                   '. Review required.')
        );
    END IF;
END$$

DELIMITER ;
```

Listing 5.1: Trigger — AlertLog on long stay

[TIP] Pro Tip

In your viva: "This trigger fires automatically on every bed allocation update. It decouples the alert logic from application code, ensuring hospital staff are notified of overstays regardless of which application inserts data."

5.2 Stored Procedure: Admit Patient

```sql
DELIMITER $$

CREATE PROCEDURE admit_patient(
    IN  p_patient_id  INT,
    IN  p_doctor_id   INT,
    IN  p_ward_id     INT,
    IN  p_visit_type  VARCHAR(20),
    OUT p_visit_id    INT
)
BEGIN
    DECLARE ward_cap   INT;
    DECLARE current_occ INT;

    -- Check ward capacity
    SELECT capacity INTO ward_cap FROM Ward WHERE ward_id =
       p_ward_id;
    SELECT COUNT(*) INTO current_occ
      FROM BedAllocation
     WHERE ward_id = p_ward_id AND discharge_date IS NULL;

    IF current_occ >= ward_cap THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Ward is at full capacity. Cannot
               admit patient.';
    ELSE
        -- Create visit record
        INSERT INTO Visit (patient_id, doctor_id, visit_date,
           visit_type)
        VALUES (p_patient_id, p_doctor_id, CURDATE(),
           p_visit_type);

        SET p_visit_id = LAST_INSERT_ID();

        -- Allocate bed
        INSERT INTO BedAllocation (patient_id, ward_id,
           admit_date)
        VALUES (p_patient_id, p_ward_id, CURDATE());
    END IF;
END$$

DELIMITER ;

-- Usage example:
-- CALL admit_patient(101, 5, 3, 'IPD', @new_visit_id);
-- SELECT @new_visit_id;
```

Listing 5.2: Stored Procedure — admit_patient

5.3 View: Doctor Performance Summary

```sql
CREATE VIEW vw_doctor_performance AS
SELECT
    d.doctor_id,
    d.specialization,
    d.experience_years,
    COUNT(DISTINCT v.visit_id)              AS total_visits,
    AVG(ba.days_stayed)                     AS avg_patient_stay,
    SUM(CASE WHEN o.status = 'Recovered'
             THEN 1 ELSE 0 END)             AS recovered_count,
    SUM(CASE WHEN o.status = 'Readmitted'
             THEN 1 ELSE 0 END)             AS readmission_count,
    ROUND(
      SUM(CASE WHEN o.status = 'Recovered' THEN 1 ELSE 0 END)
      / COUNT(DISTINCT v.visit_id) * 100, 2
    )                                       AS recovery_rate_pct
FROM Doctor d
LEFT JOIN Visit       v  ON d.doctor_id   = v.doctor_id
LEFT JOIN BedAllocation ba ON v.patient_id = ba.patient_id
LEFT JOIN Outcome      o  ON v.visit_id   = o.visit_id
GROUP BY d.doctor_id, d.specialization, d.experience_years;
```

Listing 5.3: View — doctor_performance

Chapter 6

Phase 4 — Data Simulation

Phase 4: Data Simulation — Where Projects Die

Students with empty or 10-row tables fail vivas instantly. You need volume. Here is exactly how to generate it.

6.1 Data Volume Targets

Table	Target Rows	Method
Patient	500	Python Faker / SQL loop
Doctor	50	Manual + scripted
Disease	30	ICD-10 subset (real codes)
Medicine	60	Generic drug names
Ward	10	Manual
Visit	2,000	Python random distribution
Diagnosis	2,400	1.2 diagnoses/visit avg
Treatment	3,000	1.5 treatments/visit avg
BedAllocation	800	40% IPD conversion rate
Outcome	2,000	3-class distribution

6.2 Python Data Generation Script

```python
import random
import datetime
import mysql.connector

# Connect
conn = mysql.connector.connect(
    host="localhost", user="root",
    password="your_password", database="careops_oltp"
)
cur = conn.cursor()

age_groups = ['0-17','18-30','31-45','46-60','61-75','76+']
```

```python
genders    = ['M', 'F', 'O']
blood_grps = ['A+','A-','B+','B-','AB+','AB-','O+','O-']
cities     = ['Mumbai','Delhi','Bangalore','Chennai','Kolkata',
              'Hyderabad','Pune','Ahmedabad']

#        Insert 500 patients


for _ in range(500):
    cur.execute("""
        INSERT INTO Patient (age_group, gender, blood_group, city)
        VALUES (%s, %s, %s, %s)
    """, (random.choice(age_groups), random.choice(genders),
          random.choice(blood_grps), random.choice(cities)))

conn.commit()

#        Insert 2000 visits (randomized over 2 years)

start_date = datetime.date(2022, 1, 1)
visit_types = ['OPD','IPD','Emergency']
weights     = [0.6, 0.3, 0.1]

for _ in range(2000):
    patient_id = random.randint(1, 500)
    doctor_id  = random.randint(1, 50)
    rand_days  = random.randint(0, 730)
    visit_date = start_date + datetime.timedelta(days=rand_days)
    vtype      = random.choices(visit_types, weights=weights)[0]
    cur.execute("""
        INSERT INTO Visit (patient_id, doctor_id, visit_date,
            visit_type)
        VALUES (%s, %s, %s, %s)
    """, (patient_id, doctor_id, visit_date, vtype))

conn.commit()

#        Insert outcomes (3-class distribution)

statuses = ['Recovered','Improved','Readmitted','Deceased']
weights  = [0.55, 0.30, 0.12, 0.03]

cur.execute("SELECT visit_id, visit_date FROM Visit")
visits = cur.fetchall()

for visit_id, visit_date in visits:
    status       = random.choices(statuses, weights=weights)[0]
    outcome_date = visit_date + datetime.timedelta(days=random.randint
        (1, 30))
    cur.execute("""
        INSERT INTO Outcome (visit_id, status, outcome_date)
        VALUES (%s, %s, %s)
    """, (visit_id, status, outcome_date))

conn.commit()
cur.close()
conn.close()
print("Data generation complete.")
```

Listing 6.1: data_generator.py — Simulated Patient + Visit Data

[!] Brutal Honesty

If you use Python Faker or any random generator: **seed the random number generator** (`random.seed(42)`) so your data is reproducible. An examiner asking "run it again" should get the same data. If you cannot reproduce your data, you will look incompetent.

Chapter 7

Phase 5 — Data Warehouse Design

Phase 5: Data Warehouse — The DW Component

This is where most students copy a star schema from the internet and cannot explain it. You will be able to explain every table and column here because you designed it yourself.

7.1 Granularity Declaration

Grain: One row in `Fact_Treatment` represents the aggregate treatment activity for a **disease** – **doctor** – **day** combination.

[TIP] Pro Tip

You MUST state the grain explicitly in your report and viva. Grain is the single most important design decision in a data warehouse. If you cannot state the grain, the examiner will deduct heavily.

7.2 Star Schema DDL

```sql
CREATE DATABASE IF NOT EXISTS careops_dw;
USE careops_dw;

--
-- DIMENSION: DATE
--

CREATE TABLE Dim_Date (
    date_id       INT          PRIMARY KEY,   -- format: YYYYMMDD
```

```sql
    full_date     DATE        NOT NULL,
    day_of_week   VARCHAR(10),
    day_num       INT,
    month_num     INT,
    month_name    VARCHAR(12),
    quarter       INT,
    year_num      INT
);

--


-- DIMENSION: DISEASE
--


CREATE TABLE Dim_Disease (
    disease_sk    INT          AUTO_INCREMENT PRIMARY KEY,  --
       surrogate key
    disease_code  VARCHAR(10) NOT NULL,
    disease_name  VARCHAR(100),
    category      VARCHAR(60),
    severity_tier VARCHAR(20)
);

--


-- DIMENSION: DOCTOR
--


CREATE TABLE Dim_Doctor (
    doctor_sk          INT  AUTO_INCREMENT PRIMARY KEY,
    doctor_id          INT  NOT NULL,
    specialization     VARCHAR(80),
    experience_band    VARCHAR(20),  -- '0-5 yrs', '6-10 yrs', etc
       .
    department         VARCHAR(60)
);

--


-- DIMENSION: WARD
--


CREATE TABLE Dim_Ward (
    ward_sk     INT  AUTO_INCREMENT PRIMARY KEY,
    ward_id     INT  NOT NULL,
```

```sql
    ward_name   VARCHAR(60),
    ward_type   VARCHAR(40),
    capacity    INT
);

--
-- DIMENSION: OUTCOME TYPE  (conformed dimension)
--

CREATE TABLE Dim_Outcome (
    outcome_sk    INT          AUTO_INCREMENT PRIMARY KEY,
    outcome_label VARCHAR(20) NOT NULL    -- Recovered, Improved,
       Readmitted
);

--
-- FACT TABLE: TREATMENT
--

CREATE TABLE Fact_Treatment (
    fact_id               BIGINT AUTO_INCREMENT PRIMARY KEY,
    date_id               INT      NOT NULL,
    disease_sk            INT      NOT NULL,
    doctor_sk             INT      NOT NULL,
    ward_sk               INT,
    outcome_sk            INT,
    total_cases           INT      NOT NULL DEFAULT 0,
    recovered_cases       INT      NOT NULL DEFAULT 0,
    readmitted_cases      INT      NOT NULL DEFAULT 0,
    avg_recovery_days     DECIMAL(6,2),
    avg_treatment_cost    DECIMAL(10,2),
    total_bed_days        INT,
    FOREIGN KEY (date_id)    REFERENCES Dim_Date(date_id),
    FOREIGN KEY (disease_sk) REFERENCES Dim_Disease(disease_sk),
    FOREIGN KEY (doctor_sk)  REFERENCES Dim_Doctor(doctor_sk),
    FOREIGN KEY (ward_sk)    REFERENCES Dim_Ward(ward_sk),
    FOREIGN KEY (outcome_sk) REFERENCES Dim_Outcome(outcome_sk)
);
```

Listing 7.1: Data Warehouse Schema — careops_dw database

7.2.1 Why Surrogate Keys?

- Surrogate keys (`disease_sk`, `doctor_sk`, etc.) decouple the warehouse from the OLTP system.

- If the OLTP `doctor_id` changes (e.g., after a system migration), the warehouse history is preserved.
- This is standard Kimball methodology — mention "Kimball" in your viva for bonus points.

Chapter 8

Phase 6 — ETL Pipeline

Phase 6: ETL — Where 80% of Students Fail

Most students either skip ETL or write it as a single INSERT...SELECT. Real ETL has three distinct stages. Implement all three and label them clearly.

8.1 ETL Architecture

8.2 Step 1: Extract

```sql
-- Run in context of careops_oltp
-- Extract base data needed for one ETL run

CREATE TEMPORARY TABLE etl_extract AS
SELECT
    v.visit_id,
    v.visit_date,
    v.doctor_id,
    d.disease_code,
    d.severity          AS diagnosis_severity,
    o.status            AS outcome_status,
    ba.days_stayed,
    (
      SELECT SUM(t2.dosage_mg * m2.unit_cost)
      FROM Treatment t2
      JOIN Medicine m2 ON t2.medicine_id = m2.medicine_id
      WHERE t2.visit_id = v.visit_id
    )                   AS treatment_cost,
    ba.ward_id
FROM Visit       v
```

```sql
JOIN Diagnosis   d  ON v.visit_id   = d.visit_id
LEFT JOIN Outcome    o  ON v.visit_id   = o.visit_id
LEFT JOIN BedAllocation ba ON v.patient_id = ba.patient_id
    AND ba.admit_date <= v.visit_date;
```

Listing 8.1: ETL Step 1 — Extract raw data from OLTP

8.3 Step 2: Transform

```sql
CREATE TEMPORARY TABLE etl_transform AS
SELECT
    DATE_FORMAT(e.visit_date, '%Y%m%d')  AS date_id,
    e.disease_code,
    e.doctor_id,
    e.ward_id,
    COUNT(*)                             AS total_cases,
    SUM(CASE WHEN e.outcome_status = 'Recovered'
             THEN 1 ELSE 0 END)          AS recovered_cases,
    SUM(CASE WHEN e.outcome_status = 'Readmitted'
             THEN 1 ELSE 0 END)          AS readmitted_cases,
    AVG(e.days_stayed)                   AS avg_recovery_days,
    AVG(e.treatment_cost)                AS avg_treatment_cost,
    SUM(e.days_stayed)                   AS total_bed_days
FROM etl_extract e
GROUP BY
    DATE_FORMAT(e.visit_date, '%Y%m%d'),
    e.disease_code,
    e.doctor_id,
    e.ward_id;
```

Listing 8.2: ETL Step 2 — Transform and aggregate

8.4 Step 3: Load

```sql
-- Run in context of careops_dw

INSERT INTO Fact_Treatment (
    date_id, disease_sk, doctor_sk, ward_sk,
    total_cases, recovered_cases, readmitted_cases,
    avg_recovery_days, avg_treatment_cost, total_bed_days
)
SELECT
    t.date_id,
    dd.disease_sk,
    dr.doctor_sk,
    dw.ward_sk,
    t.total_cases,
    t.recovered_cases,
    t.readmitted_cases,
    t.avg_recovery_days,
```

```sql
    t.avg_treatment_cost,
    t.total_bed_days
FROM careops_oltp.etl_transform t
JOIN Dim_Disease dd ON dd.disease_code = t.disease_code
JOIN Dim_Doctor  dr ON dr.doctor_id    = t.doctor_id
LEFT JOIN Dim_Ward dw ON dw.ward_id    = t.ward_id;
```

Listing 8.3: ETL Step 3 — Load into data warehouse

Chapter 9

Phase 7 — Analytics Queries

9.1 Query 1: Disease Trend Over Time

```sql
USE careops_dw;

SELECT
    dim_d.full_date,
    dis.disease_name,
    dis.category,
    SUM(ft.total_cases)    AS monthly_cases
FROM Fact_Treatment ft
JOIN Dim_Date    dim_d ON ft.date_id    = dim_d.date_id
JOIN Dim_Disease dis   ON ft.disease_sk = dis.disease_sk
GROUP BY dim_d.year_num, dim_d.month_num, dis.disease_name, dis.
   category, dim_d.full_date
ORDER BY dim_d.year_num, dim_d.month_num, monthly_cases DESC;
```

Listing 9.1: Monthly disease case volume

9.2 Query 2: Doctor Efficiency Ranking

```sql
SELECT
    dr.specialization,
    SUM(ft.total_cases)                                AS
       total_cases,
    SUM(ft.recovered_cases)                            AS recovered
       ,
    ROUND(SUM(ft.recovered_cases)
          / NULLIF(SUM(ft.total_cases),0) * 100, 2) AS
             recovery_rate_pct,
    ROUND(AVG(ft.avg_recovery_days), 2)                AS
       avg_stay_days,
    RANK() OVER (
        ORDER BY SUM(ft.recovered_cases)
                 / NULLIF(SUM(ft.total_cases),0) DESC
```

```sql
    )                                                    AS
        efficiency_rank
FROM Fact_Treatment ft
JOIN Dim_Doctor dr ON ft.doctor_sk = dr.doctor_sk
GROUP BY dr.specialization
ORDER BY efficiency_rank;
```

Listing 9.2: Doctor efficiency — recovery rate and avg stay

9.3 Query 3: Ward Utilization Rate

```sql
SELECT
    dw.ward_name,
    dw.ward_type,
    dw.capacity,
    SUM(ft.total_bed_days)                              AS
        total_bed_days_used,
    ROUND(SUM(ft.total_bed_days)
          / (dw.capacity * 365) * 100, 2)               AS
             utilization_rate_pct
FROM Fact_Treatment ft
JOIN Dim_Ward dw ON ft.ward_sk = dw.ward_sk
GROUP BY dw.ward_name, dw.ward_type, dw.capacity
ORDER BY utilization_rate_pct DESC;
```

Listing 9.3: Ward occupancy rate

9.4 Query 4: Cost Analysis by Disease

```sql
SELECT
    dis.category,
    dis.disease_name,
    ROUND(AVG(ft.avg_treatment_cost), 2)  AS avg_cost,
    ROUND(MAX(ft.avg_treatment_cost), 2)  AS max_cost,
    ROUND(MIN(ft.avg_treatment_cost), 2)  AS min_cost,
    SUM(ft.total_cases)                   AS case_volume
FROM Fact_Treatment ft
JOIN Dim_Disease dis ON ft.disease_sk = dis.disease_sk
GROUP BY dis.category, dis.disease_name
ORDER BY avg_cost DESC;
```

Listing 9.4: Average treatment cost per disease category

9.5 Query 5: Readmission Rate by Disease

```sql
SELECT
    dis.disease_name,
    dis.severity_tier,
```

```sql
    SUM(ft.total_cases)                                AS total_cases,
    SUM(ft.readmitted_cases)                           AS readmissions,
    ROUND(SUM(ft.readmitted_cases)
          / NULLIF(SUM(ft.total_cases),0) * 100, 2) AS readmission_rate_pct
FROM Fact_Treatment ft
JOIN Dim_Disease dis ON ft.disease_sk = dis.disease_sk
GROUP BY dis.disease_name, dis.severity_tier
ORDER BY readmission_rate_pct DESC
LIMIT 10;
```

Listing 9.5: Readmission analysis — the “Outcome” table payoff

Chapter 10

Phase 8 — Viva Preparation

Phase 8: Viva — Where Projects Are Won and Lost

The code is 40% of the grade. Explaining it is the other 60%. Prepare these questions cold.

10.1 Expected Viva Questions with Model Answers

Q1: Why do you need a data warehouse? Why not just query the OLTP?

Model Answer: OLTP is optimized for transactional reads and writes with row-level locking. Running aggregation queries (GROUP BY, AVG, SUM across millions of rows) on OLTP degrades performance for concurrent users. The data warehouse uses a denormalized star schema, which is optimized for read-heavy analytical queries. Additionally, the ETL process allows data cleansing and transformation before analysis, which cannot be done safely on live data.

Q2: Why a star schema? Why not snowflake?

Model Answer: A star schema has fewer joins than a snowflake schema because dimension tables are fully denormalized. For our analytical queries, query performance is more important than storage normalization in the warehouse. The star schema also maps directly to our declared grain: one fact row per disease-doctor-day combination. Snowflake would add complexity without benefit for our query patterns.

Q3: What is the grain of your fact table?

Model Answer: The grain is one row per disease, per doctor, per day. This means each row in `Fact_Treatment` aggregates all visits for a specific disease treated by a specific doctor on a specific date.

Q4: Why did you add the Outcome table?

Model Answer: Without outcome data, the system can only count cases. With outcomes, we can answer "Are patients actually recovering?" and "Which diseases have the highest readmission rates?" This transforms the project from a visit-counting system into a treatment effectiveness analytics system.

Q5: What does your trigger do?

Model Answer: The `trg_long_stay` trigger fires AFTER any UPDATE on `BedAllocation`. If the computed `days_stayed` exceeds 14, it automatically inserts an alert record into `AlertLog`. This is an example of a database-level business rule — the alert fires regardless of which application layer updates the bed record.

Q6: How is your data in 3NF?

Model Answer: 3NF requires that every non-key attribute depends on the key, the whole key, and nothing but the key. In our Visit table, no non-key attribute depends on another non-key attribute. Doctor name and specialization are in the Doctor table, not in Visit. Disease category is in the Disease table, not in Diagnosis. There are no transitive dependencies.

Chapter 11

Final Honest Assessment

[!] Brutal Honesty

What will cause you to fail this project (even with good code):

- Not being able to state the grain of your fact table on demand
- Having empty or near-empty tables during the demo
- Having OLTP data that violates 3NF
- Not being able to explain why ETL has three stages
- Saying "I used a trigger" but not knowing what event fires it
- Having the Outcome table in the schema but no data in it

[TIP] Pro Tip

What will make you the best project in the batch:

- Citing "Kimball methodology" when explaining the star schema
- Demonstrating the trigger firing live with a 15+ day stay update
- Showing the readmission rate query and interpreting the output
- Having clean ER and star schema diagrams
- Being able to define grain, surrogate key, and conformed dimension without notes

11.1 Recommended Build Order

1. Create OLTP schema (`careops_oltp`) with all DDL
2. Run Python data generation script (seed=42)
3. Implement trigger and stored procedure
4. Create views for OLTP analytics
5. Create DW schema (`careops_dw`)
6. Run ETL (Extract → Transform → Load)
7. Execute all 5 analytics queries and save output

8. Draw ER diagram (use draw.io or MySQL Workbench)
9. Draw star schema diagram
10. Prepare viva answers
11. Write report (this document is your report structure)

Estimated Final Grade: 9.5 / 10

If all phases are implemented and you can defend every design decision.